A memristor-based stochastic number generator for low-cost arithmetic

with an image processing case study

Tanner Robison
Electrical and Computer Engineering
Portland State University
Portland, Oregon
tnr@pdx.edu

Ivan Flores Suares
Electrical and Computer Engineering
Portland State University
Portland, Oregon
ivanf@pdx.edu

Ren Cadua
Electrical and Computer Engineering
Portland State University
Portland, Oregon
rencadua@pdx.edu

***Abstract*—Edge computing devices face substantial area, power, and reliability constraints, though most of their workloads, like image processing or neural network inferencing, tolerate small computational errors, making them suitable for stochastic computing (SC). SC encodes values as the probability of a 1 appearing in a random bitstream. Arithmetic with this is simplified to single gates, with multiplication being done with an AND gate, and scaled addition with a multiplexer. Individual bit errors have only a minor effect on the result. The stochastic number generators (SNGs) that produce the random bitstreams usually dominate the cost of these systems, reducing the validity of the advantages. This paper proposes using a behavioral SNG model using the memristor stochastic switching model proposed by Naous et al., in which pseudorandom values are compared with the voltage-dependent switching probability. This presents the applied voltage as a control knob for setting the stochastic value. We will pair the SNG with a multiplier and scaled adder and evaluate the circuit in 2 ways: the encoding accuracy of the voltage-to-probability mapping and the arithmetic error as a function of bitstream length. An additional stretch goal of the project will be to implement the unit in an image processing case study to reveal the effects of progressive precision and graceful degradation.**

***Index Terms*—stochastic, memristors, arithmetic, image processing, SystemVerilog, FPGA**

I. Introduction

Current edge computing devices, like IoT, wearables, or biomedical implants, rely more and more on local processing for stronger real-time capabilities but face limited area, power, and reliability constraints. Many of the workloads these devices work with are error tolerant, like image processing, neural network inferencing, or signal filtering, meaning they accept small numerical errors without changing the result. Conventional binary arithmetic wastes area and power computing exact results that these workloads don't need, while facing larger consequences from bit errors. Stochastic computing presents itself as a useful computing paradigm for these types of workloads by way of simplified arithmetic gate construction. Arithmetic like multiplication and addition can be done with single gates. The circuitry needed to generate stochastic streams is the current hang-up with the paradigm and usually dominates the cost of these systems, offsetting the advantages of the simpler arithmetic.

Memristors are non-volatile devices whose resistances depend on an internal state variable. The changing of this resistance is known as the switching characteristic and is inherently probabilistic. Naous et al. [18] modeled this behavior as a voltage-dependent switching probability. This maps a voltage to a probability, which presents the voltage as a natural control knob for setting a stochastic value.

This paper proposes using this voltage-dependent switching probability to create a behavioral Stochastic Number Generator (SNG) model implemented on an FPGA. The model will work by comparing pseudorandom values with the $P_{switch}(V)$ equation, allowing for the applied voltage control knob. The SNG will be paired with a scaled adder and multiplier to perform arithmetic on these streams, and the device will be evaluated in 2 ways: the encoding accuracy of the voltage-to-probability mapping and the arithmetic error as a function of the bitstream length.

II. Background

A. Stochastic Computing

Stochastic computing (SC) is a hardware-level paradigm of probabilistic computing. Numerical values are encoded as the probability of a 1 appearing in a random bitstream [13], and a bit's position in the stream does not matter. This is what makes SC suitable for error-tolerant workloads: a wrong bit in binary has exponential consequences as the bit works up to the most significant bit position. Because a bit's position in the stream does not matter, this consequence does not exist. This allows for complex arithmetic to be executed with simpler logic gates than conventional binary operations. For example, multiplying two stochastic bitstreams takes just one gate: for unipolar numbers [0,1], this is an **AND** gate, and for bipolar numbers [-1,1], this is an **XNOR** gate [14]. To do the same operation in positional binary would require many gates. Addition is done with a 2:1 multiplexer and requires a 3rd stochastic stream. The select for the MUX must be supplied with the new stream

strictly with a value of 0.5 [14]. Given input streams A and B, the output is 1 with probability [14]

$$0.5\,p_A + 0.5\,p_B = (p_A + p_B)/2 \tag{1}$$

The result is scaled by $1/2$ because a probability can't exceed 1, so the sum of 2 values must be mapped back into that range. The MUX circuit can perform scaled addition for both unipolar and bipolar numbers. Because each MUX stage halves the result, a value that passes through k cascaded additions must be multiplied by 2^k after converting back to binary to get the true value.

Converting back to binary requires a counter that counts the number of 1s over L cycles, where L is the bitstream length. The trade-off comes with time, because getting N bits of precision from a bitstream requires 2^N clock cycles, or more once you account for things like correlation and noise. This results in latency growing exponentially as precision grows, where 8-bit precision requires 256 cycles, but 16-bit requires 65,536. Low gate count does not necessarily mean low energy per result, because energy is the product of power and computation time; energy per result also scales exponentially with precision. With these considerations, stochastic computing is mainly suited for highly parallel, error-tolerant, and low-precision workloads [17]. Some example applications where this is useful are neural network accelerators [8], FIR filters [16], biomedical signal processing [6], image processing [4], audio filtering [7], etc.

B. Correlation

In statistics, correlation is the tendency of two random variables to move together, so that knowing the value of one provides some information about the other. In stochastic computing, correlation describes how the positions of the 1s in two bitstreams line up, independent of how many 1s each stream contains. This matters because the function a gate computes depends on that alignment: multiplication and scaled addition are only computed correctly when their input streams, including the MUX select stream, are statistically independent [1]. For example, one might try to square a stream encoding 0.5 by feeding it into both inputs of an AND gate, but the result is 0.5 rather than 0.25, because each bit is simply ANDed with itself [2]. Delaying one copy does not necessarily solve the problem: delaying the stream 01010101 by one cycle produces 10101010, and the AND gate now outputs 0. This failure is caused by *autocorrelation*, the correlation between a stream and a time-shifted copy of itself, which is high for periodic streams.

Ordinary correlation is poorly suited to bitstreams, because its achievable range depends on the values the streams encode. Alaghi and Hayes introduced the stochastic computing correlation (SCC) measure [1], which compares the observed overlap of 1s, $P(A \wedge B)$, with the overlap expected for independent streams, $p_A p_B$, normalized by the maximum or minimum overlap possible:

$$\mathrm{SCC} = \begin{cases} \dfrac{P(A \wedge B) - p_A p_B}{\min(p_A, p_B) - p_A p_B}, & P(A \wedge B) > p_A p_B \\ \dfrac{P(A \wedge B) - p_A p_B}{p_A p_B - \max(p_A + p_B - 1,\, 0)}, & \text{otherwise} \end{cases} \tag{2}$$

An SCC of 0 indicates independent streams, $+1$ indicates the maximum possible overlap, and -1 the minimum. Correlation is not always harmful: with $\mathrm{SCC} = +1$, an AND gate computes $\min(p_A, p_B)$ and an XOR gate computes $|p_A - p_B|$ [1]. In practice, independent streams are produced by driving each SNG with its own LFSR. Because LFSRs that share a feedback polynomial produce time-shifted copies of the same sequence, accuracy improves when each LFSR uses a different feedback polynomial as well as a different seed [5].

C. Stochastic Number Generators

A stochastic number generator (SNG) converts an n-bit binary value X into a stochastic bitstream. The conventional design pairs a random number source with an n-bit binary comparator. Each clock cycle, an n-bit random number R is compared with X, and the SNG outputs a 1 if $R < X$ and a 0 otherwise, so each output bit is 1 with probability $X/2^n$. The random source is typically a maximal-length n-bit linear feedback shift register (LFSR), which cycles pseudo-randomly through $2^n - 1$ states using only n flip-flops and a few XOR gates. Because the accuracy of stochastic arithmetic depends on the quality of these streams, the LFSR's seed and feedback polynomial measurably affect computational accuracy [5]. Low-discrepancy sequences can replace the LFSR to make streams converge faster, at the cost of additional hardware [3].

Although the arithmetic in a stochastic system may be a single gate, every independent input stream requires its own SNG, consisting of an n-bit register, feedback logic, and an n-bit comparator. As a result, SNGs rather than the arithmetic logic usually dominate the cost of stochastic systems: CMOS-based SNGs have been reported to consume over 80% of a stochastic system's power and area [11], and up to 80% of total design area [15]. Sharing one random source among several SNGs reduces this cost, but it introduces correlation between the resulting streams, which breaks operations such as multiplication that require independent inputs. This trade-off between SNG cost and stream independence motivates the search for SNGs built from emerging devices with intrinsic randomness, such as memristors.

D. Memristors

A memristive device is a circuit element whose electrical behavior depends on an internal state that is influenced by its previous electrical excitation. The memristor was originally proposed in theory by Chua in 1971 [9] as a relationship between charge and magnetic flux. It was later generalized by Chua and Kang [10] to memristive systems, in which the device resistance can depend on one or more internal state

variables. In 2008, Strukov et al. [19] identified a physical device exhibiting this behavior.

A voltage-controlled memristive system can generally be represented as

$$i(t) = G(x, v, t)v(t) \tag{3}$$

$$\frac{dx}{dt} = f(x, v, t) \tag{4}$$

where x represents the internal device state, G is the state-dependent conductance, and f describes how the state changes in response to an applied voltage. The state dependence distinguishes the system from an ordinary resistor, where

$$i(t) = \frac{v(t)}{R} \tag{5}$$

In a memristor, an applied voltage can alter the device state, changing the conductance. This can be good because it means we can program the resistance at any time, but it also means the electrical operation (voltage pulse) used to evaluate a computational result can disturb the value stored by the device.

Memristor switching is also random. Apply the same voltage pulse many times, and the device will switch on some tries but not others. How often it switches depends on the pulse voltage magnitude, with higher voltages switching more often [18]. This randomness is a problem for memory, but it is exactly what stochastic computing needs: pulsing a memristor once per clock cycle produces a random bitstream whose fraction of 1s is set by the voltage [12].

III. Proposed Design & Methodology

A. FPGA Design

The system will be implemented in SystemVerilog and deployed on a Basys 3 (Artix-7) FPGA using Vivado, clocked by the board's 100 MHz oscillator. Fig. 1 shows the datapath. Because the FPGA contains no memristive devices, the SNG is a behavioral emulation: it reproduces the voltage-to-probability encoding of the stochastic switching model of Naous et al. [18], while a pseudorandom source supplies the randomness that a physical device would provide intrinsically.

The probability that a write pulse of amplitude V and width Δt switches the memristor is

$$P_{switch}(V) = 1 - e^{-\Delta t/\tau(V)}, \qquad \tau(V) = \tau_0\, e^{-V/V_0}, \tag{6}$$

with variables defined in the Appendix. We use the fitted values $\tau_0 \approx 2.69 \times 10^5$ s and $V_0 \approx 0.163$ V from [18] and a pulse width of $\Delta t = 1$ ms. Each clock cycle emulates one write pulse followed by an ideal reset, so successive output bits are independent.

Each SNG pairs a 16-bit LFSR with its own lookup table (LUT) and a comparator. Every LUT has 2048 entries, one per voltage code, and entry k stores the 16-bit threshold $T_k = \mathrm{round}\big(P_{switch}(V_k)\,(2^{16}-1)\big)$. At the start of a run, the SNG's voltage code selects its threshold, and each cycle the SNG outputs a 1 when the LFSR value is less than or equal to that threshold, so the fraction of 1s in the stream approximates $P_{switch}(V_k)$. Each LUT is generated from slightly perturbed device parameters, emulating device-to-device variation. The LUTs are generated by a Python script and loaded into block RAM at synthesis using `$readmemh`; all three together occupy 96 Kbit of the Basys 3's 1.8 Mbit of block RAM.

With $\Delta t = 1$ ms and a margin for device variation, the 2048 codes span approximately 1.4 V to 4.0 V, a resolution of about 1.28 mV per code. This range extends beyond the 2.6–3.6 V measurements used to fit the model, so switching probabilities near either end of the range are extrapolated [18].

The design contains three SNGs. Two produce the operand streams a and b, and the third produces the adder's select stream s, using the entry of its own LUT closest to 0.5. Each LFSR uses a distinct feedback polynomial and seed so that the three streams are independent, and its period of 65,535 cycles is longer than the longest bitstream. Multiplication is a single AND gate and scaled addition a 2:1 multiplexer, both implemented directly in FPGA logic.

Counters accumulate the 1s on each SNG stream and on both arithmetic outputs over L cycles, with L ranging from 2^4 to 2^{15}, and return the counts over UART to a Python host script. The host computes each SNG's encoding accuracy by comparing its measured fraction of 1s with $P_{switch}(V_k)$, and the arithmetic error by comparing the AND and MUX outputs with the exact product and scaled sum. Voltage codes are chosen from each device's own LUT, so the measured error reflects the stochastic computation rather than device variation.

A control finite-state machine (FSM) coordinates each run in five states. In IDLE, it waits for a start byte from the host. In RECEIVE, it collects a fixed 14-byte packet containing the three voltage codes, three LFSR seeds, and the bitstream length L, and stores them in configuration registers; a timeout returns it to IDLE if the packet is incomplete. In LOAD, each SNG reads its threshold from its LUT, the LFSRs are loaded with their seeds, and the counters are cleared. In RUN, the SNGs, arithmetic gates, and counters operate for L clock cycles. In REPORT, the five 16-bit counts are returned to the host as 10 bytes, and the FSM returns to IDLE. A run takes about 2.5 ms, dominated by UART transfer time, so sweeping all 2048 voltage codes at one bitstream length takes about 5 s.

B. Image Processing Case Study

As a stretch goal, the arithmetic unit will be applied to grayscale images. Rather than storing images on the FPGA, the host will stream them over the same UART link one pixel at a time: it converts each 8-bit intensity to a voltage code, the FPGA runs the stochastic operation for L cycles, and the returned count becomes the output pixel. Scaled addition blends two images, and multiplication applies a mask or brightness scale. At 115,200 baud, a 128×128 image takes on the order of ten seconds per pass, which is acceptable for an offline study. Repeating each operation at increasing L demonstrates progressive precision, with output quality measured by Peak Signal-to-Noise Ratio (PSNR) against a floating-point reference.

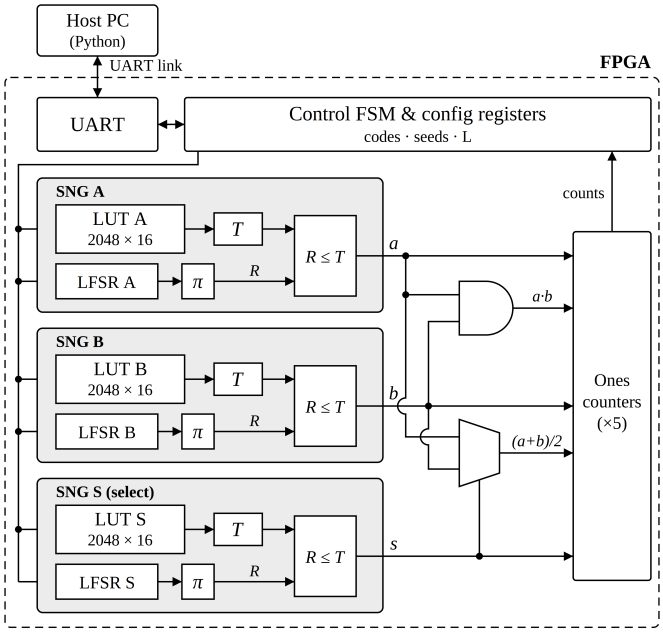


Fig. 1. Datapath. Each SNG's voltage code indexes that SNG's own P_{switch} LUT, generated from perturbed device parameters. The returned thresholds are latched and compared with independent LFSRs to produce stochastic streams, which drive the AND multiplier and MUX adder. Ones are counted over L cycles and reported to the host over UART. The Pi blocks represent the LFSRs' bit shuffles, and the T blocks represent the LUTs' results.

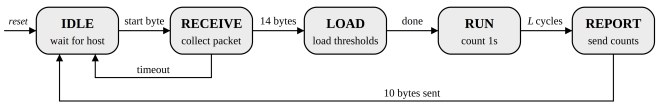


Fig. 2. Block diagram of the FSM controller. Each run receives a 14-byte configuration packet, loads thresholds and seeds, runs the SNGs for L cycles, and returns the five counts. An incomplete packet times out back to IDLE.

IV. Timeline

- **Week 4:** Python switching model, LUT generation, start on LFSR and SNG RTL.
- **Week 5:** Complete week 4 work, control FSM, and UART RTL; verify in simulation against the Python model.
- **Week 6:** Synthesize for the Basys 3; complete host script and test it against a Python emulation of the FPGA. Begin board bring-up if hardware is delivered.
- **Week 7:** Board bring-up; confirm hardware counts match simulation; run encoding-accuracy and arithmetic-error sweeps.
- **Week 8:** Analysis and figures; image case study if on schedule.
- **Week 9:** Cleanup & polishing.
- **Week 10:** Final Report.

V. Conclusion

Stochastic computing reduces arithmetic to single logic gates, but the stochastic number generators (SNGs) that feed those gates dominate the cost of stochastic systems. This project proposes an SNG built on the voltage-dependent switching probability of memristors, using the stochastic switching model of Naous et al. [18] so that the applied voltage directly sets the value a bitstream encodes. The SNG will be emulated in SystemVerilog on a Basys 3 FPGA, with a separate lookup table for each device to model device-to-device variation and an LFSR standing in for the device's intrinsic randomness. Three such SNGs will drive an AND-gate multiplier and a MUX-based scaled adder, and a control FSM will run each experiment and report results to a host computer over UART.

The design will be evaluated on two measures: how accurately each SNG reproduces its device's switching probability, and the error of the multiplier and adder as a function of bitstream length. We expect both errors to fall roughly as $1/\sqrt{L}$, halving with each fourfold increase in bitstream length. Because the FPGA emulates memristors rather than containing them, this work does not measure hardware savings; instead, it tests whether voltage-controlled switching probability can serve as an accurate encoding for stochastic arithmetic, a prerequisite for building such SNGs from physical devices. If time permits, an image processing case study will demonstrate progressive precision, with output quality improving as bitstream length increases.

Acknowledgment

The authors declare that generative AI was used for proofreading and editing assistance in the preparation of this manuscript. Figures 1 and 2 use AI-generated illustrations. Some AI-generated content was included in the Proposed Design & Methodology section to aid in more detailed project planning and content comprehension.

Appendix

Switching Model Variables

Equation (6):

$$P_{switch}(V) = 1 - e^{-\Delta t/\tau(V)}, \qquad \tau(V) = \tau_0\, e^{-V/V_0}$$

P_{switch} – Probability that one voltage pulse switches the memristor.
V – Applied pulse voltage.
Δt – Pulse width.
$\tau(V)$ – Characteristic switching time at voltage V.
τ_0 – Characteristic switching time at 0 V.
V_0 – Voltage increase that shortens τ by a factor of e.
e – Euler's number, 2.718.

Peak Signal-to-Noise Ratio

PSNR measures how close an output image is to a reference image. Higher values mean a closer match.

$$\mathrm{PSNR} = 10\log_{10}\left(\frac{MAX^2}{MSE}\right)$$

$$MSE = \frac{1}{MN}\sum_{i=1}^{M}\sum_{j=1}^{N}\big(I(i,j) - K(i,j)\big)^2$$

PSNR – Peak signal-to-noise ratio, in decibels (dB).
MAX – Largest possible pixel value, 255 for 8-bit grayscale.
MSE – Mean squared error between the two images.
M, N – Image height and width in pixels.
$I(i, j)$ – Reference pixel value, computed exactly in floating point.
$K(i, j)$ – Pixel value produced by the stochastic hardware.

REFERENCES

[1] Armin Alaghi and John P. Hayes. Exploiting correlation in stochastic circuit design. In *2013 IEEE 31st International Conference on Computer Design (ICCD)*, pages 39–46, 2013.

[2] Armin Alaghi and John P. Hayes. Survey of stochastic computing. *ACM Trans. Embed. Comput. Syst.*, 12(2s), May 2013.

[3] Armin Alaghi and John P. Hayes. Fast and accurate computation using stochastic circuits. In *Proceedings of the Design, Automation and Test in Europe Conference (DATE)*, 2014.

[4] Armin Alaghi, Cheng Li, and John P. Hayes. Stochastic circuits for real-time image-processing applications. In *Proceedings of the 50th Annual Design Automation Conference (DAC)*, pages 136:1–136:6, Austin, TX, USA, 2013.

[5] Jason H. Anderson, Yuko Hara-Azumi, and Shigeru Yamashita. Effect of lfsr seeding, scrambling and feedback polynomial on stochastic computing accuracy. In *2016 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 1550–1555, March 2016.

[6] Timothy J. Baker and John P. Hayes. CeMux: Maximizing the accuracy of stochastic mux adders and an application to filter design. *ACM Transactions on Design Automation of Electronic Systems*, 27:1–26, 2022.

[7] Timothy J. Baker, Yiqiu Sun, and John P. Hayes. Benefits of stochastic computing in hearing aid filterbank design. In *Proceedings of the IEEE Biomedical Circuits and Systems Conference (BioCAS)*, pages 1–5, 2021.

[8] Bradley D. Brown and Howard C. Card. Stochastic neural computation I: Computational elements. *IEEE Transactions on Computers*, 50(9):891–905, 2001.

[9] L. Chua. Memristor—the missing circuit element. *IEEE Transactions on Circuit Theory*, 18(5):507–519, 1971.

[10] L. O. Chua and S. M. Kang. Memristive devices and systems. *Proceedings of the IEEE*, 64(2):209–223, 1976.

[11] João Paulo C. de Lima et al. All-in-memory stochastic computing using ReRAM. In *Proceedings of the 62nd ACM/IEEE Design Automation Conference (DAC)*, pages 50–56, 2025. arXiv:2504.08340.

[12] Siddharth Gaba, Patrick Sheridan, Jiantao Zhou, Shinhyun Choi, and Wei Lu. Stochastic memristive devices for computing and neuromorphic applications. *Nanoscale*, 5(13):5872–5878, 2013.

[13] B. R. Gaines. Stochastic computing. In *Proceedings of the April 18-20, 1967, Spring Joint Computer Conference*, AFIPS '67 (Spring), page 149–156, New York, NY, USA, 1967. Association for Computing Machinery.

[14] Brian Gaines. Stochastic computing systems. *Adv Inf Syst Sci*, 2, 01 1969.

[15] Saransh Gupta, Mohsen Imani, Joonseop Sim, Andrew Huang, Fan Wu, M. Hassan Najafi, and Tajana Rosing. SCRIMP: A general stochastic computing architecture using ReRAM in-memory processing. In *Proceedings of the Design, Automation and Test in Europe Conference (DATE)*, 2020.

[16] Yin Liu and Keshab K. Parhi. Lattice FIR digital filter architectures using stochastic computing. In *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 1027–1031, 2015.

[17] Rajit Manohar. Comparing stochastic and deterministic computing. *IEEE Computer Architecture Letters*, 14(2):119–122, 2015.

[18] Rawan Naous, Maruan Al-Shedivat, and Khaled Nabil Salama. Stochasticity modeling in memristors. *IEEE Transactions on Nanotechnology*, 15(1):15–28, 2016.

[19] Dmitri B. Strukov, Gregory S. Snider, Duncan R. Stewart, and R. Stanley Williams. The missing memristor found. *Nature*, 453(7191):80–83, May 2008.